

Allocator-aware library wrappers for dynamic allocation

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Example use cases](#)
4. [Dynamic allocation wrappers](#)
5. [Unique pointers](#)
6. [Design questions](#)
7. [Acknowledgements](#)
8. [Proposed wording](#)

Revision history

- [P0211R0](#): Initial proposal, pre-Jacksonville 2016.
- D0211r1: make pointer in `allocate_delete` const; delete unnecessary null check from `allocate_new`; add some more design questions.
- [P0211R1](#): Rename functions as requested by LEWG in Jacksonville 2016. Add conversions to `allocation_deleter`. Add formal wording.
- [P0211R2](#): Remove `const` from a few places where it would not have been allowed. Use `std::to_address` (introduced by [P0653R2](#)). Add `noexcept` to `allocation_deleter` constructors. Allow rebinding. Wording tweaks.
- P0211R3 (this version): Discuss the absence of `_default_init` versions. Update *Remarks* to *Constraints*, and update section numbers.

Summary

Short form: We propose to add library functions that allow the systematic use of allocators as a customisation point for dynamic allocations. The new functions complete the following picture:

	using operator {new,delete}	using allocator
Manual	<code>T * p = new T(args...)</code>	<code>auto p = allocator_new<T>(alloc, args...)</code> new!
	<code>delete p</code>	<code>allocator_delete(alloc, p)</code> new!
Unique pointer	<code>default_delete<T></code>	<code>allocation_deleter<T></code> new!
	<code>make_unique<T>(args...)</code>	<code>allocate_unique<T>(alloc, args...)</code> new!
Shared pointer	<code>make_shared<T>(args...)</code>	<code>allocate_shared<T>(alloc, args...)</code>

Long form: The standard library rarely uses `new/delete` directly, but instead allows customisation of dynamic allocation and object construction via allocators. Currently this customisation is only available for container *elements* and for `shared_ptr` (as well as for a few other types that require dynamically allocated memory), but not for the top-level objects themselves.

The proposal is to complete the library facilities for allocator-based customisation by providing a direct mechanism for creating and destroying a dynamically stored object through an allocator, as well as a new deleter type for `unique_ptr` to hold an allocator-managed unique pointee, together with the appropriate factory function.

Example use cases

- Consider an arena allocator. A vector may put its elements into the arena, but the vector itself cannot easily be placed in the same arena. The proposed `std::allocate_unique<std::vector<T, ScopedArenaAlloc<T>>>(arena_alloc)` allows this. (Here we assume the use of the usual alias idiom `template <class T> using ScopedArenaAlloc = std::scoped_allocator_adaptor<ArenaAlloc<T>>;`).
- Consider a shared memory allocator. As in the previous example, containers will put their elements in shared memory, but one process needs to create the container itself and place it in shared memory. This is now possible with `auto p = std::allocator_new<std::vector<T, ScopedShmemAlloc<T>>>(shmem_alloc)`. The returned pointer is presumably an offset pointer, and its offset value needs to be communicated to the other participating processes. The last process to use the container calls `allocator_delete(shmem_alloc, p)`.
- Allocator support is [frequently requested on Stack Overflow](#).

Dynamic allocation wrappers

Allocation and object creation

```
template <class T, class A, class ...Args>
auto allocator_new(A& alloc, Args&&... args) {
    using TTraits = typename allocator_traits<A>::template rebind_traits<T>;
    using TAlloc = typename allocator_traits<A>::template rebind_alloc<T>;

    auto a = TAlloc(alloc);
    auto p = TTraits::allocate(a, 1);

    try {
        TTraits::construct(a, to_address(p), std::forward<Args>(args)...);
        return p;
    } catch(...) {
        TTraits::deallocate(a, p, 1);
        throw;
    }
}
```

Object destruction and Deallocation

```
template <class A, class P>
void allocator_delete(A& alloc, P p) {
    using Elem = typename pointer_traits<P>::element_type;
    using Traits = typename allocator_traits<A>::template rebind_traits<Elem>;

    Traits::destroy(alloc, to_address(p));
    Traits::deallocate(alloc, p, 1);
}
```

Unique pointers

To allow `std::unique_ptr` to use custom allocators, we first need a deleter template that stores the allocator:

```
template <class A>
struct allocation_deleter {
    using pointer = typename allocator_traits<A>::pointer;

    A a_; // exposition only
};
```

```
allocation_deleter(const A& a) noexcept : a_(a) {}

void operator()(pointer p) {
    allocator_delete(a_, p);
}
};
```

The factory function is:

```
template <class T, class A, class ...Args>
auto allocate_unique(A& alloc, Args&&... args) {
    using TAlloc = typename allocator_traits<A>::rebind_alloc<T>;
    return unique_ptr<T, allocation_deleter<TAlloc>>(allocator_new<T>(alloc,
std::forward<Args>(args)...), alloc);
}
```

The type `T` must not be an array type. The pathological case where the template argument of `allocation_deleter<A>` is equal (up to qualification) to `allocation_deleter<A>` has to be taken into account for the purpose of the constructor.

Design questions

Naming

The name for the deleter class: “`allocation_deleter`” or “`allocation_delete`”? (We had previously used “`allocate_delete`” for both the function and the class, but that would have been very confusing.)

Default initialization

In C++20, the creation functions `make_unique`, `make_shared`, and `allocate_shared` were augmented by `make_unique_default_init`, `make_shared_default_init`, and `allocate_shared_default_init`. Whereas the former (called with no arguments) value-initialize an object (e.g. `::new T()`), the latter default-initialize instead (e.g. `::new T`).

One might ask whether there should not also be analogous default-initializing versions of the allocation helpers that are proposed here. However, this is a false analogy, and the answer is no: The proposed allocation helpers are meant to be a symmetric pair of allocation/construction and destruction/deallocation. Note that `allocator_delete` and `allocation_deleter::operator()` both call the allocator’s `destroy` function, which therefore must be matched with initialization by `construct`. This in turn precludes any notion of default-initialization, which the allocator API does not allow.

By contrast, `make_unique` and `make_shared` and their default-initializing versions do not use allocators and can thus correctly call `delete` or `delete[]` regardless of how the initialization was performed, and `allocate_shared/allocate_shared_default_init` use different dynamic deleters so that the allocator is used for the former, and raw `delete` is used for the latter.

Acknowledgements

Many thanks to Daniel Krügler for a thorough review and invaluable corrections of the first version from 2016, to the members of LEWG who reviewed the draft and provided many valuable improvements in 2018, and to Alisdair Meredith for pointing out default-initialization.

Proposed wording

In [memory.syn, 20.10.2], add to the synopsis:

```
// 20.10.8, uses_allocator
template <class T, class Alloc> struct uses_allocator;

[...]

// 20.10.9, allocator traits
template <class Alloc> struct allocator_traits;

// 20.10.?, allocation helpers:
template <class A, class ...Args>
typename allocator_traits<A>::pointer allocator_new(A& alloc, Args&&... args);
template <class A>
void allocator_delete(A& alloc, typename allocator_traits<A>::pointer p);

// 20.10.10?, the default allocator:
template <class T> class allocator;
template <> class allocator<void>;
template <class T, class U>
bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
template <class T, class U>
bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;
```

```
// 20.11.1 class template unique_ptr:
template <class T> struct default_delete;
template <class T> struct default_delete<T[]>;
template <class A> struct allocation_deleter;
template <class T, class D = default_delete<T>> class unique_ptr;
template <class T, class D> class unique_ptr<T[], D>;

template <class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
template <class T> unique_ptr<T> make_unique(size_t n);
template <class T, class... Args> unspecified make_unique(Args&&...) = delete;
template <class T, class A, class ...Args> unique_ptr<T, allocation_deleter<A>>
allocate_unique(A& alloc, Args&&... args)
```

Insert a new subsection between 20.9.9 (allocator traits) and 20.9.10 (the default allocator):

20.10.? Allocation helpers [allocator.alloc_helpers]

The function templates `allocator_new` and `allocator_delete` create and delete objects dynamically using an allocator to provide storage and perform construction (rather than by looking up an allocation function ([basic.stc.dynamic.allocation, 6.6.4.4.1])).

```
template <class T, class A, class ...Args>
    typename allocator_traits<A>::pointer allocator_new(A& alloc, Args&&... args);
```

Requires: A shall satisfy the *Cpp17Allocator* requirements ([allocator.requirements, 16.5.3.5]).

Effects: Let `TTraits` be the type `allocator_traits<A>::rebind_traits<T>`. Obtains storage for one element from a copy `a` of `alloc` rebound for type `T` by calling `TTraits::allocate(a, 1)` and constructs an object by calling `TTraits::construct(a, to_address(p), std::forward<Args>(args)...) , where p is the pointer obtained from the allocation. If the construction exits with an exception, the storage is released using TTraits::deallocate(a, p, 1).`

Returns: A pointer to the obtained storage that holds the constructed object. [Note: This pointer may have a user-defined type. – end note]

Throws: Any exception thrown by `TTraits::allocate` or by `TTraits::construct`.

```
template <class A, class P>
    void allocator_delete(A& alloc, P p);
```

Requires: A shall satisfy the *Cpp17Allocator* requirements ([allocator.requirements, 16.5.3.5]). P shall satisfy the *Cpp17NullablePointer* requirements ([nullablepointer.requirements, 16.5.3.3]). The pointer p was obtained from an allocator that compares equal to alloc, and p is dereferenceable.

Effects: Let `TTraits` be the type `allocator_traits<A>::rebind_traits<pointer_traits<P>::element_type>`. Uses a copy a of alloc rebound to the `TTraits::value_type` to destroy the object *p by calling `TTraits::destroy(a, to_address(p))` and to release the underlying storage by calling `TTraits::deallocate(a, p, 1)`.

Insert a new subsection 20.11.1.1.? after 20.11.1.1.3 (`default_delete<T[]>`):

20.11.1.1.? allocation_deleter<A> [unique.ptr.dltr.alloc]

```
template <class A>
struct allocation_deleter {
    using pointer = typename allocator_traits<A>::pointer;

    allocation_deleter(const A& alloc) noexcept;

    template <class B>
    allocation_deleter(const allocation_deleter<B>& other) noexcept;

    void operator()(pointer p);

private:
    [[no_unique_address]] A a_; // exposition only
};
```

```
allocation_deleter(const A& alloc) noexcept;
```

Effects: Initializes a_ with alloc.

Constraints: A, ignoring qualifications, is not allocation_deleter<A>.

```
template <class B>
    allocation_deleter(const allocation_deleter<B>& other) noexcept;
```

Effects: Initializes a_ with other.a_.

Constraints: `typename allocator_traits<B>::pointer` is implicitly convertible to `pointer`.

```
void operator()(pointer p);
```

Effects: Calls `allocator_delete(a_, p)`.

Append a new paragraph to the end of subsection 20.11.1.4 (`unique_ptr` creation):

```
template <class T, class A>
    unique_ptr<T, allocation_deleter<allocator_traits<A>::rebind_alloc<T>>>
    allocate_unique_default_init(A& alloc)
```

Constraints: T is not an array.

Returns: `unique_ptr<T, allocation_deleter<allocator_traits<A>::rebind_alloc<T>>>`
`(allocator_new<T>(alloc, std::forward<Args>(args)...), alloc).`

Throws: Any exception thrown by `allocator_new<T>`.